Java syntax and program style

CSC103 Programming Fundamentals / Lecture 03

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Rewrite: int a=6;int b=4;System.out.println(2*(a+b));
Use names, spacing and a useful comment.

Rename a and b so that their roles are visible.

Learning objectives

- Distinguish tokens, identifiers and reserved words
- Explain syntax versus meaning
- Apply consistent names, indentation and comments

CLO-1
Tokens and special symbols
Identifiers and reserved words
Comments and documentation
Syntax, semantics and style

Tokens and special symbols

- Tokens include keywords, identifiers, literals and operators.
- Braces delimit blocks. Parentheses group expressions and arguments. A semicolon ends many statements.

Example

```java
int score = 75;
System.out.println(score);
```

Identifiers and reserved words

- Choose meaningful names such as totalMarks.
- Names are case-sensitive and cannot be reserved words.
- Class names conventionally use PascalCase and variables use camelCase.

Example

```
Valid: studentCount, marks2
Invalid: 2marks, class
Different: score and Score
```

Comments and documentation

- A line comment starts with //. A block comment uses /* ... */.
- Documentation comments use /** ... */ before declarations.
- Useful comments explain intent or constraints.

Example

```java
// Convert whole hours to minutes.
int totalMinutes = hours * 60;
```

Syntax, semantics and style

- Syntax defines legal structure. Semantics concerns what the program means.
- Indentation helps people see nesting. Descriptive names help people understand intent.

Example

```
int length = 5;
int width = 3;
int area = length * width;
```

A statement can be legal and still be wrong

- The compiler accepts an expression that satisfies Java grammar and type rules.
- Correctness also requires the expression to match the intended quantity.
- Names and units help a reader check this connection.

Example

```
For sides 6 cm and 4 cm:
Area = 6 * 4 = 24 cm^2
Perimeter = 2 * (6 + 4) = 20 cm
```

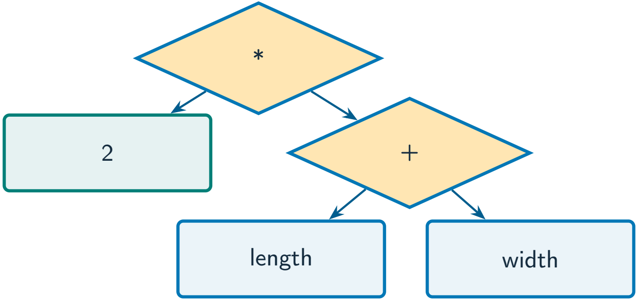
Documentation that explains decisions

- A purpose comment states what the program calculates.
- A constraint comment explains the assumptions behind a calculation.
- Changing code and leaving an old comment creates misleading documentation.

Example

```
Useful: Side lengths are in centimetres.
Weak: Add length and width.
The expression already shows the addition.
```

Read an expression as a syntax tree



What to notice

In 2 * (length + width), the grouped sum supplies one operand of multiplication.

Key distinctions

Token	Category	Purpose
int	Keyword	Declares an integer type
perimeter	Identifier	Names the result
2	Literal	Supplies a fixed value
*	Operator	Multiplies operands
;	Separator	Ends the declaration statement

These distinctions determine which operation or control structure fits the problem.

First example: execution

```java
// A rectangle with sides in centimetres.
int length = 5;
int width = 3;
int area = length * width;
System.out.println(area);
```

First example: result

15
All variable names describe their purpose.
The comment identifies the unit of measurement.

Guided practice

Rewrite: int a=6;int b=4;System.out.println(2*(a+b));
Use names, spacing and a useful comment.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Rename a and b so that their roles are visible.
- Group length + width before doubling it.
- Use spacing, separate statements and a comment that identifies the measurement unit.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```java
public class Solution03 {
  public static void main(String[] args) throws Exception {
    // Side lengths are in centimetres.
    int length = 6;
    int width = 4;
    int perimeter = 2 * (length + width);
    System.out.println(perimeter + " cm");
  }
}
```

Execution trace

Expression	Meaning	Value
length	First side	6 cm
width	Second side	4 cm
length + width	Adjacent sides	10 cm
2 * (length + width)	All four sides	20 cm

Follow the changing state in execution order.

Solution result

20 cm

Why this result follows
Use spacing, separate statements and a comment that identifies the measurement unit.

A common incorrect approach

int class = 20;
System.out.println(Class);

Example

```
class is reserved. Replace both names with a consistent identifier, for example classSize.
```

Boundary and failure checks

Check the stated input assumptions.
Format your previous program. Add one purpose comment and explain why each variable name is meaningful.

// Side lengths are in centimetres.
int length = 6;
int width = 4;
System.out.println(2 * (length + width));
Output: 20.

Check your understanding

Is total marks a legal variable name? Does indentation change the meaning of braces?

Write a prediction and explain the rule that supports it.

Answer and explanation

No: a space separates tokens. No: braces define the block, while indentation communicates that structure to readers.

class is reserved. Replace both names with a consistent identifier, for example classSize.

Compare cases and predict outcomes

Expression	length=6, width=4	Interpretation
2 * (length + width)	20	Perimeter
2 * length + width	16	Only length is doubled
length * width	24	Area

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Rewrite an unclear perimeter expression using named variables, parentheses and a units label. Explain why area and perimeter need different units.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```java
int length = 6;
int width = 4;
int perimeter = 2 * (length + width);
System.out.println(perimeter + " cm");
```

- The parentheses make the sum one operand of multiplication.
- Perimeter is a length, measured here in cm; area uses square cm.
- The output is 20 cm.

Repairing the Welcome program

- The course uses public static void main(String[] args) as its entry point. Main and main are different identifiers.
- A string literal needs matching double quotes; a character literal cannot hold a sentence.
- Balance class and method braces, then use indentation to expose their nesting.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: Repairing the Welcome program

Task

Repair the source activity containing public void Main and an incorrectly quoted greeting. Print the greeting once.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture03 {
  public static void main(String[] args) throws Exception {
    System.out.println("Welcome to Java!");
  }

}
```

Worked trace and expected result

Fault	Correction	Why
Main	main	Names are case-sensitive
Missing static	Add static to main	Conventional launch entry point
Mismatched quotes	Use double quotes	The greeting is a String

Expected console output

```
Welcome to Java!
```

Extend the ISB example

Practice

Extend the corrected program to print the greeting five times without using loops yet.

Explain your reasoning

Repeat the println statement five times. A later loop can express the repetition more compactly.

Lecture summary

- tokens, identifiers and reserved words
- syntax versus meaning
- consistent names, indentation and comments

- Rename a and b so that their roles are visible.
- Group length + width before doubling it.
- Use spacing, separate statements and a comment that identifies the measurement unit.

After-class practice

Format your previous program. Add one purpose comment and explain why each variable name is meaningful.

Syllabus reading
Liang Ch 2
Next lecture: Variables and console input